MemoBot User Manual

Cover Page

Project Title: MemoBot - Memory Layer for Robots
Version Number: 1.0.0
Team Members: Jason Yang, Zeyi Chen, Yuxin Zhang, Yeary Yuan **Instructor name:** Luyao Niu, Mark Licata **Submission Date:** March 12, 2026

Product Overview

What is MemoBot?
MemoBot is a semantic memory storage and retrieval system designed specifically for humanoid robots. It acts as an artificial hippocampus, allowing robots to continuously record their environment and securely store those memories.

The Problem It Solves
Currently, most robots live purely in the present moment. They struggle to remember where objects were placed, who they interacted with, or what happened in the past. MemoBot bridges this gap by providing an intuitive memory layer, giving robots the context they need to interact more naturally with humans.

Who It Is Designed For
MemoBot is designed for robotics researchers, developers building assistive humanoid robots, and end-users who require an intelligent, context-aware robotic assistant in their homes or workplaces.

Primary Features

- **Continuous Multimodal Ingestion:** Captures video, audio, and physical actions continuously.
- **Semantic Search:** Uses advanced AI to allow natural language querying (e.g., "Where did I put my keys?").
- **Rich Context Retrieval:** Returns relevant video clips, text events, and detected objects.
- **Generative Answers:** Synthesizes past memories into natural, conversational responses using Large Language Models (LLMs).

System Requirements

MemoBot is a hybrid software/hardware system.

Software Requirements:

- **Supported Operating Systems:** Linux (Ubuntu 20.04+ recommended), macOS, or Windows (with WSL2).
- **Hardware Requirements:** Minimum 8GB RAM (16GB recommended), multi-core CPU.
- **Dependencies:** Docker, Docker Compose, Python 3.9+, and `ffmpeg`.
- **Internet Requirements:** A stable broadband internet connection is required for cloud

API integrations (Twelve Labs and OpenAI).
- **Required User Accounts:**
  - OpenAI API Account
  - Twelve Labs API Account

Hardware Requirements (Robot Integration):

- **Peripherals:** An onboard camera (e.g., `/dev/video0`) and microphone.
- **Power Requirements:** Standard 110-240V AC for the host server running the API. The robot operates on its respective battery specifications.
- **Environmental Conditions:** The host server must be kept in a climate-controlled, dry environment.
- **Compatibility Constraints:** The robot must be capable of executing Python scripts and running `ffmpeg` locally.

Package Contents:

- MemoBot API Server Source Code
- MemoBot Python SDK
- Docker Configuration Files
- User and Architecture Documentation

Installation and Setup Instructions

Follow these step-by-step procedures to deploy the MemoBot system.

Step 1: Install `uv`

We use uv for fast and reliable Python environment management.

```
# On Mac/Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Step 2: Clone the Repository

Open your terminal and clone the MemoBot repository to your local machine or server.

```
git clone https://github.com/your-org/memobot.git
cd memobot
```

Step 3: Configure Environment Variables

Create a configuration file to store your database credentials and API keys.

1. Copy the example environment file or create a new `.env` file in the root directory.
2. Fill in the required values:

```
# .env
PICOVOICE_ACCESS_KEY=your_picovoice_key # Required for wake word detect
GOOGLE_API_KEY=your_google_api_key      # Required for Gemini Live inte
OPENAI_API_KEY=sk-...                   # For embeddings/text fallback
TWELVE_LABS_API_KEY=tlk_...             # For video ingestion (if enabl
```

Step 4: Install Dependencies

```
# Sync dependencies and create a managed virtual environment
uv sync

# Activate the virtual environment
source .venv/bin/activate
```

Operating Instructions

Ensure you have activated your virtual environment before running the server or mock client on your Mac/Linux host.

Starting the System

1. Start the Server (Mac / Linux)

The master server handles all intelligence and routes audio back to the client.

```
# Start the server with real-time Gemini Live interaction
python memobot/robot/mac_master_v10.py --realtime

# Start the server with both real-time interaction AND memory ingestion
python memobot/robot/mac_master_v10.py --realtime --ingest
```

2. Start the Client

Option A: Running on a Physical Robot Copy the project files to the robot. The client script streams its camera and microphone directly to the master server.

```
# On the robot (e.g. NAO)
python memobot/robot/robot_client_v6.py --mac-ip <YOUR_MAC_SERVER_IP>
```

Option B: Running a Mock Client (Mac / Linux) If you don't have a robot, you can use the mock client, which will use your computer's built-in webcam and microphone.

```
# In a new terminal on your Mac/Linux machine
source .venv/bin/activate

# Run the mock client pointing to your Mac Server IP (e.g., localhost)
python memobot/robot/mock_audio_client_v2.py --host 127.0.0.1
```

Shutting Down the System

1. **Robot/Mock Client:** Safely terminate the client Python script.
2. **Server:** Safely terminate the `mac_master_v10.py` Python script.

Technical Specifications

Software Architecture: MemoBot uses a client-server architecture. The robot acts as a lightweight client capturing media and playing audio, while the heavy lifting (video

processing, active speaker detection, hotword detection, LLM streaming) is done by a Python master server.

- **Server:** `mac_master_v10.py` (Mac/Linux host)
- **Client:** `robot_client_v6.py` (Physical robot) or `mock_audio_client_v2.py` (Local computer)

Database Information (Legacy/Ingest mode):

- **Vector Store:** PostgreSQL with `pgvector` for text and metadata embeddings.
- **Graph Storage:** Knowledge Graph via Bolt/Neo4j interface.

Integrations:

- **Gemini Live:** Used for real-time conversational streaming audio.
- **Picovoice:** Used for Wake word detection.
- **Twelve Labs:** Used for generating multimodal video and audio embeddings (Ingest mode).
- **OpenAI:** Used for text embeddings and generating conversational LLM answers (Fallback/Ingest mode).

Hardware Specifications (Reference Robot):

- **Processor Requirements:** ARM or x86 processor capable of h.264 video encoding.
- **Sensors:** Minimum 720p RGB camera; omnidirectional microphone array.

Safety and Privacy

Hardware Safety Warnings

- **Mechanical Risks:** Ensure the robot's camera and movement systems are not obstructed during continuous recording.
- **Electrical Risks:** Do not expose the robot or the host server to water or extreme temperatures. Follow standard electrical safety guidelines when charging the robot.

Data Privacy and Security

- **Continuous Recording:** Because MemoBot continuously records audio and video, users must be informed that they are being recorded. Do not deploy the system in environments where there is an expectation of strict privacy (e.g., bathrooms).
- **API Security:** Never commit your `.env` file containing API keys to public version control.
- **Network Security:** The FastAPI server should not be exposed directly to the public internet without implementing a reverse proxy (like Nginx) and proper authentication mechanisms.

Troubleshooting

Problem	Possible Cause	Solution

ModuleNotFoundError	Environment not activated	Run `source .venv/bin/activate` in the root directory.
Cannot Connect / Connection Refused	Port conflicts or wrong IP	Ensure ports `50005`, `50006`, `50007`, `50009`, and `50010` are open and you provided the correct host IP.
No Audio Output on Mock Client	Wrong audio device	Run `python memobot/robot/mock_audio_client_v2.py --list-output-devices` to find your speaker index, then pass it with `--output-device <index>`.
Poor search results	Invalid API keys or no internet	Verify that `OPENAI_API_KEY`, `GOOGLE_API_KEY`, and `TWELVE_LABS_API_KEY` are correct and active in the `.env` file.

Limitations

- **Known Constraints:** Video ingestion relies on complete 5-second video chunks. Real-time streaming on a per-frame basis is not currently supported by the Twelve Labs indexing engine.
- **Performance Limitations:** The speed of memory retrieval is heavily dependent on internet bandwidth and latency to external cloud APIs.
- **Unfinished Features:** A fully localized, offline mode (without reliance on OpenAI or Twelve Labs) is planned for a future release but is not available in Version 1.0.0.